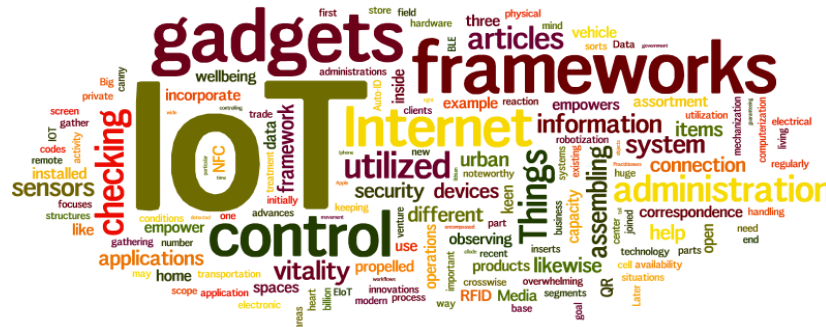


CS578: Internet of Things



MQTT: Message Queuing Telemetry Transport



Dr. Manas Khatua

Assistant Professor, Dept. of CSE, IIT Guwahati

E-mail: manaskhatua@iitg.ac.in

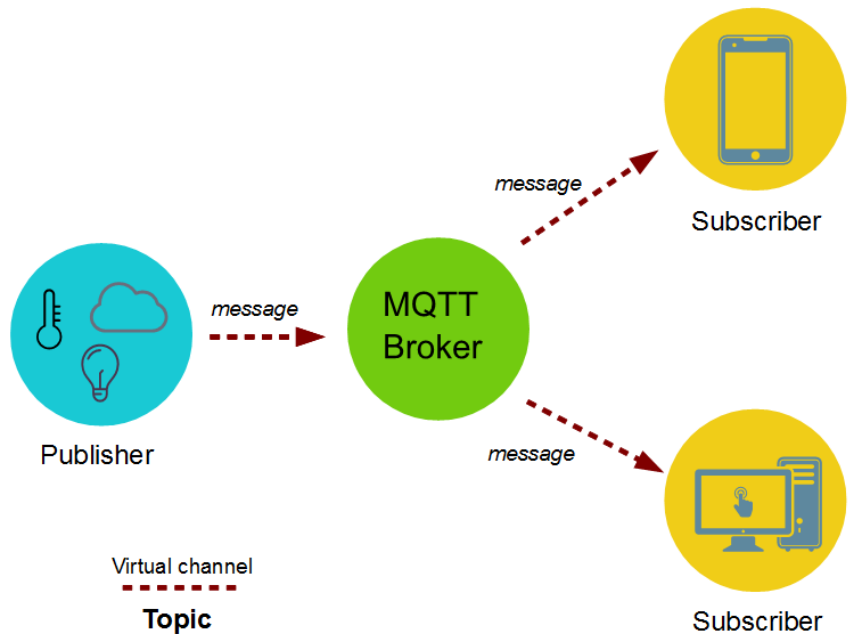
“Lead us from unreal to real. From darkness into light. From death to immortality, Om Shanti Shanti Shanti.”

What is MQTT?

- MQTT : Message Queueing Telemetry Transport protocol.
- **Reliable, Lightweight, Cost-effective protocol**
- Suited for the **transport of telemetry data** (sensor and actor data),

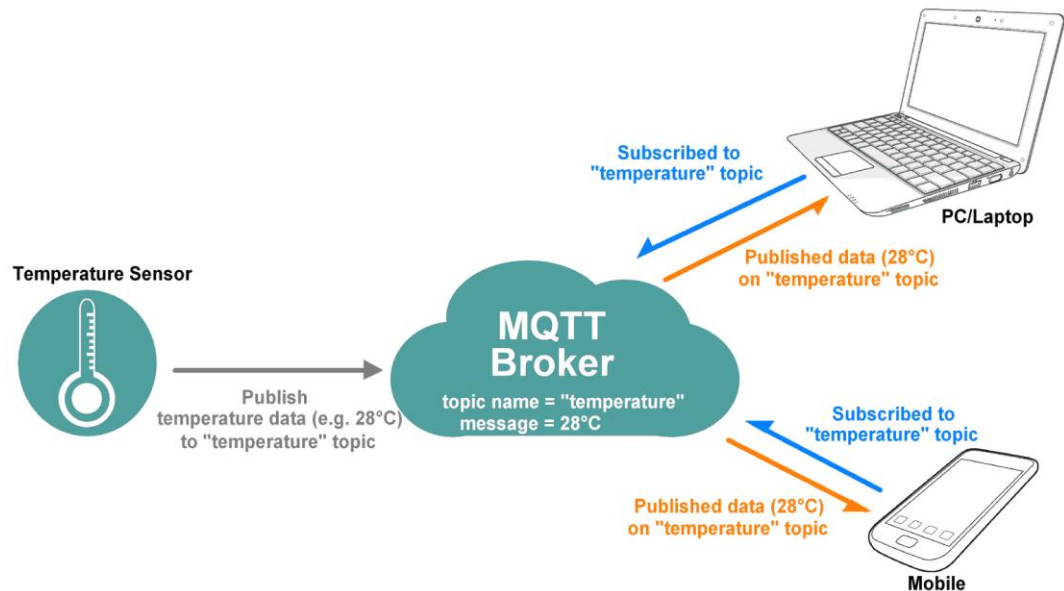
Example:

- **Light sensor** continuously sends sensor data to the **broker**.
 - **Building control application** receives sensor data from the broker and decides to activate **Camera**.
 - **The application** sends an activation message to the **camera** node (i.e. actuator) through the **broker**.
-
- Invented by Andy Stanford Clark of IBM and Arlen Nipper of Arcom (now Eurotech) in 1999
 - Used by real-life applications:
 - Amazon Web Services (AWS),
 - IBM WebSphere MQ,
 - Microsoft Azure IoT,
 - Facebook Messenger,
 - etc.



MQTT Characteristics

In 2014, MQTT was adopted and published as an official standard by OASIS (Organization for the Advancement of Structured Information Standard)



- **Asynchronous communication model** with messages (events)
- Low overhead (**2 bytes header**) for low network bandwidth applications
- **Publish / Subscribe** (PubSub) model
- **Decoupling** of data producer (publisher) and data consumer (subscriber) through topics (message queues)
- Runs on connection-oriented transport (**TCP**). To be used in conjunction with **6LoWPAN** (TCP header compression)
- MQTT mainly designed for M2M communication

Publish & Subscribe Messaging



Terminology

- A producer sends (**publishes**) a message (**publication**) on a topic (**subject**)
- A consumer **subscribes** (makes a subscription) for messages on a topic (**subject**)
- A message **server / broker** matches publications to subscriptions

Who will get the message ?

- If **no matches** the message is **discarded**
- If **one or more matches** the message is delivered to **each** matching subscriber/consumer

Topic

- A topic forms the namespace in hierarchical with each “**sub topic**” separated by a /
- An **example** topic space :
 - **A house publishes information about itself on:**
`<country>/<region>/<town>/<postalcode>/<house>/energyConsumption`
`<country>/<region>/<town>/<postalcode>/<house>/solarEnergy`
 - **And subscribes for control commands:**
`<country>/<region>/<town>/<postalcode>/<house>/thermostat/setTemp`

Wildcards

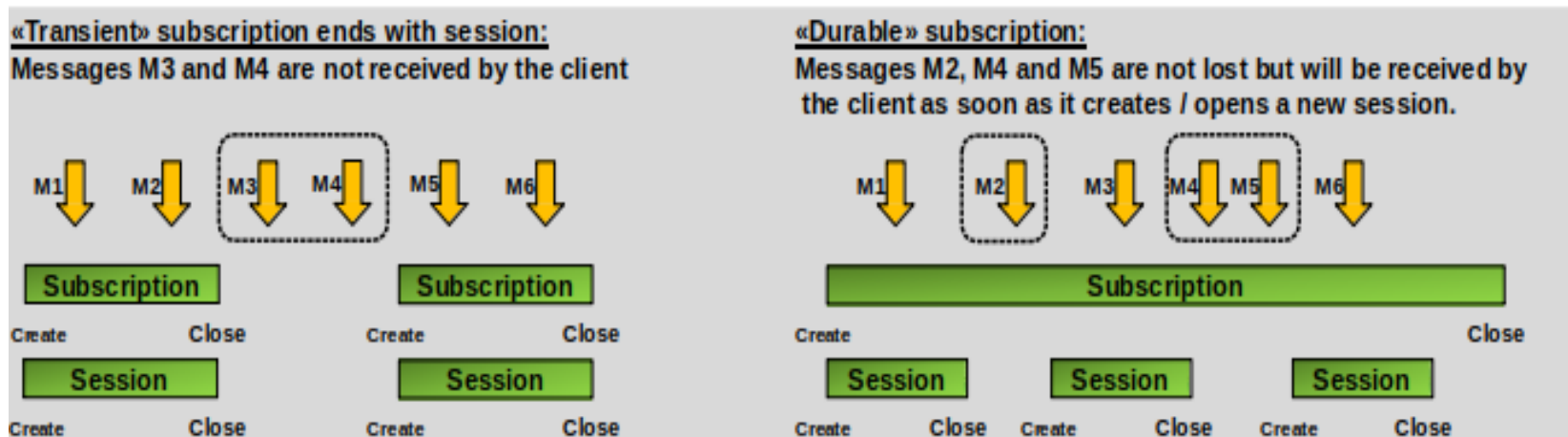
- A **subscriber** can subscribe to an **absolute topic** / can use **wildcards**:
 - **Single-level wildcards** “+” can appear anywhere in the topic string
For example:
 - Energy consumption **for 1 house** in Hursley
 - *UK/Hants/Hursley/SO212JN/1/energyConsumption*
 - Energy consumption **for all houses** in Hursley
 - *UK/Hants/Hursley/+/+/energyConsumption*
 - **Multi-level wildcards** “#” must appear **at the end** of the string
For example:
 - Details of energy consumption, solar and alarm **for all houses in SO212JN**
 - *UK/Hants/Hursley/SO212JN/#*

NOTE :

- Wildcards must be **next to a separator**
- **Cannot** be used wildcards when **publishing**

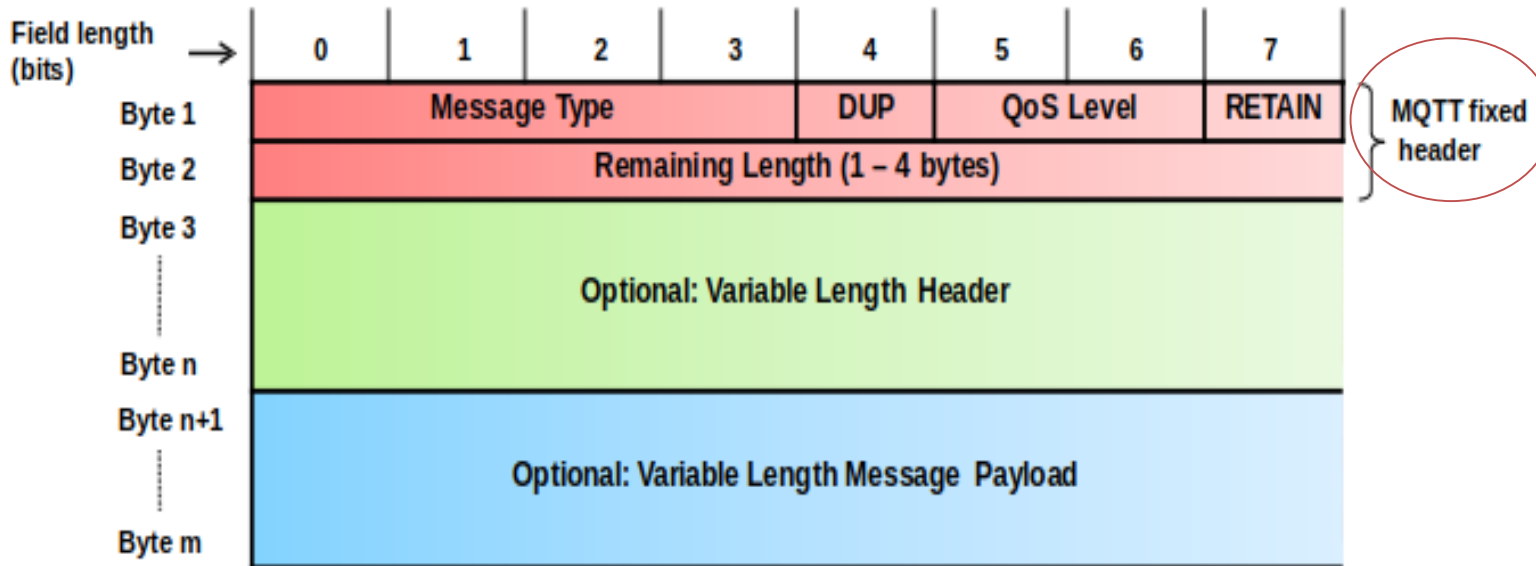
Cont...

- A subscription can be durable or non durable
- **Durable:**
 - Once a **subscription** is in place a **broker** will forward matching messages to the **subscriber**:
 - **Immediately** if the subscriber is connected.
 - If the subscriber is not connected messages **are stored on the server/broker** until the next time the subscriber connects.



- **Non-durable / Transient** (i.e. subscription ends with client session):
 - The **subscription lifetime** is the same as the time the subscriber is connected to the server / broker

MQTT Message Format



- **Message Type:** identifies the kind of MQTT packet within a message
- **DUP:** Duplicate flag – indicates whether the packet has been sent previously or not
- **QoS Level:** it allows to select different QoS level
- **Retain:** this flag notifies the server to hold onto the last received PUBLISH message data
- **Remaining Length:** specifies the size of optional fields
- MQTT is lightweight
 - because each packets consists of a **2-byte fixed header** with **optional variable header fields** and **optional payload**

Message header field	Description / Values	
Message Type 14 message types, 2 are reserved	0: Reserved	8: SUBSCRIBE
	1: CONNECT	9: SUBACK
	2: CONNACK	10: UNSUBSCRIBE
	3: PUBLISH	11: UNSUBACK
	4: PUBACK	12: PINGREQ
	5: PUBREC	13: PINGRESP
	6: PUBREL	14: DISCONNECT
	7: PUBCOMP	15: Reserved
DUP	Duplicate message flag. Indicates to the receiver that this message may have already been received. 1: Client or server (broker) re-delivers a PUBLISH, PUBREL, SUBSCRIBE or UNSUBSCRIBE message (duplicate message).	
QoS Level	Indicates the level of delivery assurance of a PUBLISH message. 0: At-most-once delivery, no guarantees, «Fire and Forget». 1: At-least-once delivery, acknowledged delivery. 2: Exactly-once delivery.	
RETAIN	1: Instructs the server to retain the last received PUBLISH message and deliver it as a first message to new subscriptions.	
Remaining Length	Indicates the number of remaining bytes in the message, i.e. the length of the (optional) variable length header and (optional) payload.	

Message Types

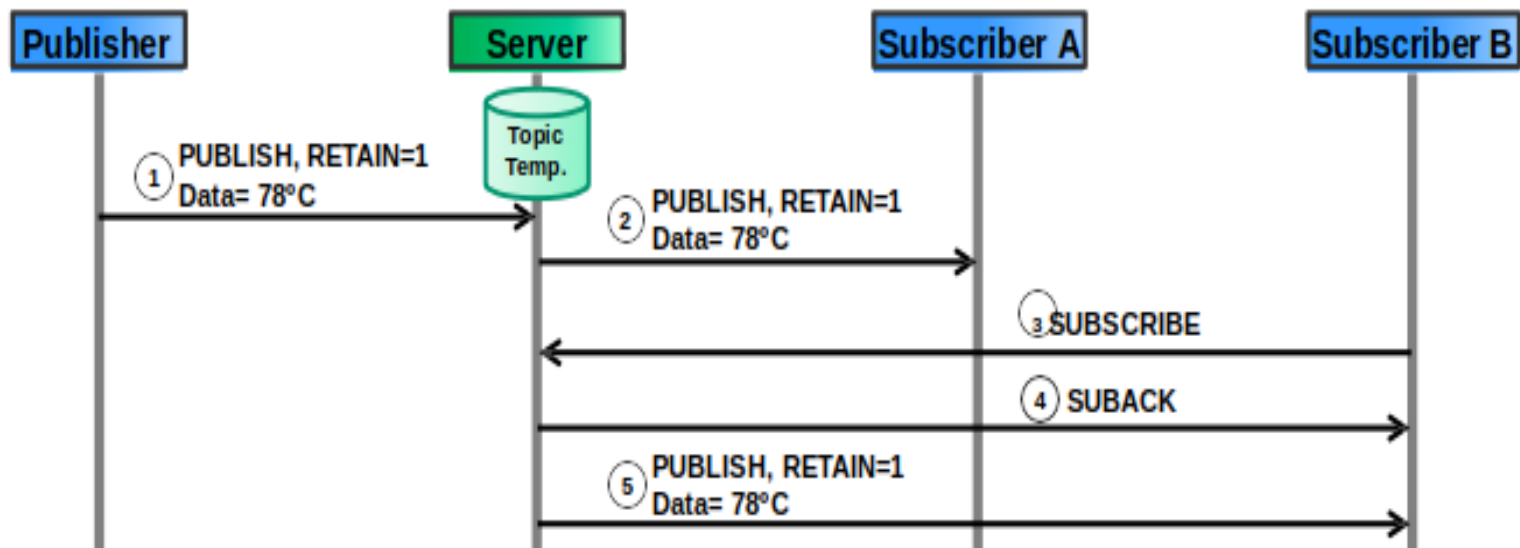


Name	Value	Direction	Description	Flags
Reserved	0	Forbidden		
CONNECT	1	$C \rightarrow S$	Client connection request	0000
CONNACK	2	$C \leftarrow S$	Server response to CONNECT	0000
PUBLISH	3	$C \leftrightarrow S$	Message publishing	Varying
PUBACK	4	$C \leftrightarrow S$	PUBLISH acknowledgement	0000
PUBREC	5	$C \leftrightarrow S$	PUBLISH received	0000
PUBREL	6	$C \leftrightarrow S$	PUBLISH release	0010
PUBCOMP	7	$C \leftrightarrow S$	PUBLISH complete	0000
SUBSCRIBE	8	$C \rightarrow S$	Client subscription to topic	0010
SUBACK	9	$C \leftarrow S$	Server response to SUBSCRIBE	0000
UNSUBSCRIBE	10	$C \rightarrow S$	Client unsubscription to a topic	0010
UNSUBACK	11	$C \leftarrow S$	Server response to UNSUBSCRIBE	0000
PINGREQ	12	$C \rightarrow S$	Keepalive PING request	0000
PINGRESP	13	$C \leftarrow S$	Keepalive PING response	0000
DISCONNECT	14	$C \rightarrow S$	Client is disconnecting	0000
Reserved	15	Forbidden		

RETAIN

RETAIN=1 in a **PUBLISH** message instructs the server to keep the message for this topic. When a new client subscribes to the topic, the server sends the retained message quickly.

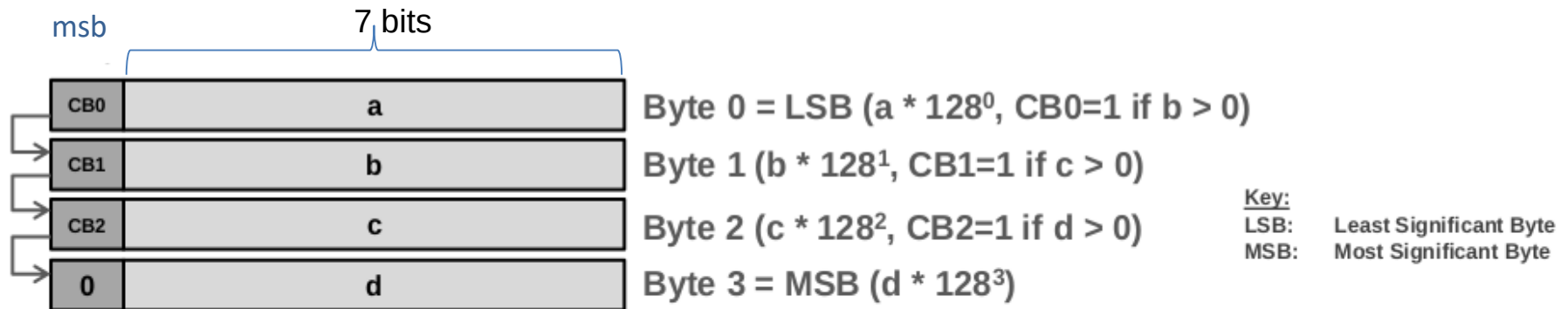
- **Typical application scenarios:**
 - Clients publish only **changes** in data, so subscribers receive the last known good value.
- **Example:**
 - Subscribers receive **last known temperature value** from the temperature data topic.
 - **RETAIN=1** indicates to subscriber B that the message may have been published some time ago.



Remaining length (RL)

- The remaining length field encodes the **sum of the lengths** of:
 - (Optional) variable length header
 - (Optional) payload
- To save bits, remaining length is a **variable length field** with 1...4 bytes.
- The **most significant bit (msb)** of a **length field** byte has the meaning **continuation bit (CB)**.
- If more bytes follow, it is set to 1.

Remaining length is encoded as:: $a * 128^0 + b * 128^1 + c * 128^2 + d * 128^3$
and placed into the RL field bytes as follows:

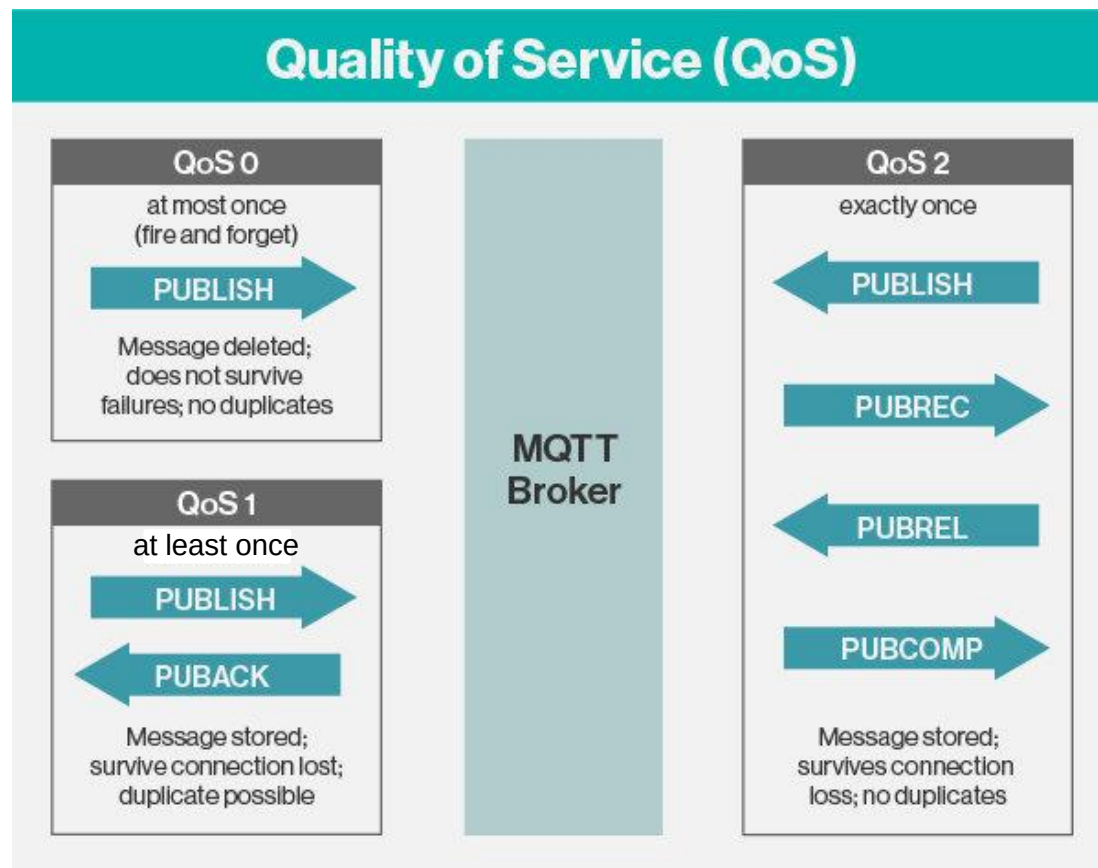


Example 1: $RL = 364 = 108 * 128^0 + 2 * 128^1 \rightarrow a=108, CB0=1, b=2, CB1=0, c=0, d=0, CB2=0$

Example 2: $RL = 25'897 = 41 * 128^0 + 74 * 128^1 + 1 * 128^2 \rightarrow a=41, CB0=1, b=74, CB1=1, c=1, CB2=0, d=0$

MQTT QoS

- Even though TCP/IP provides guaranteed data delivery, **data loss** can still occur if a **TCP connection breaks down** and **messages in transit are lost**.
- Therefore, MQTT adds **3 quality of service (QoS)** levels on top of TCP



QoS level 0:

- **At-most-once delivery** («best effort»).
- Messages are **delivered** according to the delivery guarantees of the underlying network (TCP/IP).
 - *Example application:* Temperature sensor data which is regularly published. **Loss of an individual value** is not critical since applications (i.e. consumers of the data) will anyway **integrate the values over time**

QoS level 1:

- **At-least-once delivery.**
- Messages are **guaranteed to arrive**, but there may be **duplicates**.
 - *Example application:* A door sensor senses the door state. It is important that door state changes (closed->open, open->closed) are **published losslessly** to subscribers (e.g. alarming function). Applications simply discard duplicate messages by evaluating the message ID field.

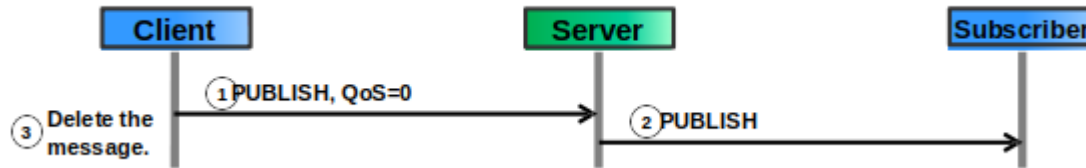
QoS level 2:

- **Exactly-once delivery.**
- This is the highest level that also incurs **most overhead** in terms of control messages and the need for **locally storing** the messages.
- **Exactly-once** is a **combination of at-least-once** and **at-most-once** delivery guarantee.
 - *Example application:* Applications where duplicate events could lead to incorrect actions, e.g. sounding an alarm as a reaction to an event received by a message. So, it **avoids duplicate**.

PUBLISH msg flow

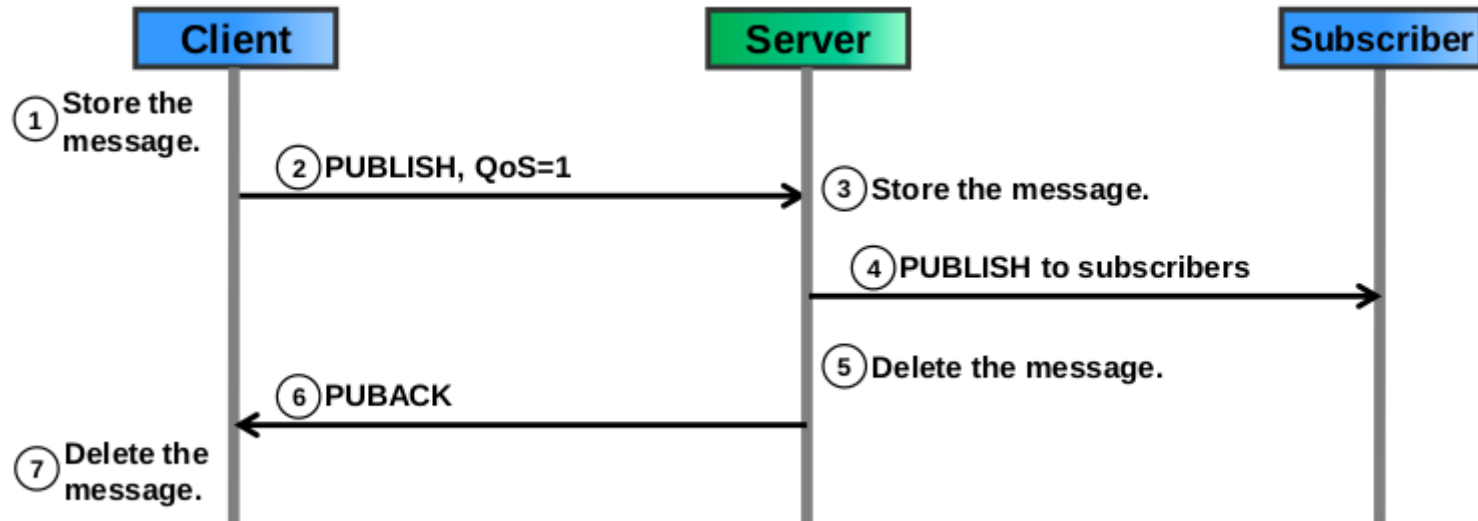
QoS level 0:

- With QoS level 0, a message is delivered with **at-most-once delivery semantics** («fire-and-forget»).



QoS level 1:

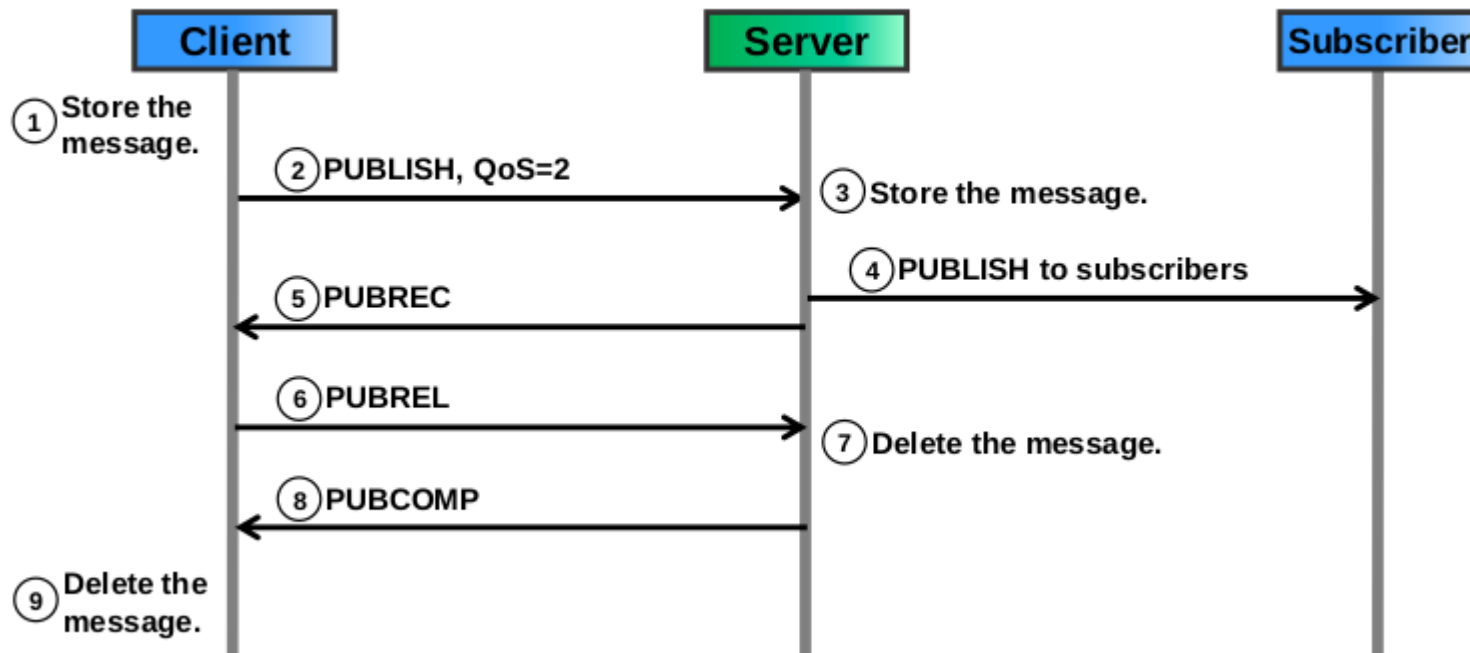
- QoS level 1 affords **at-least-once delivery semantics**. If the client does not receive the **PUBACK** in time, it **re-sends** the message.



Cont...

QoS level 2:

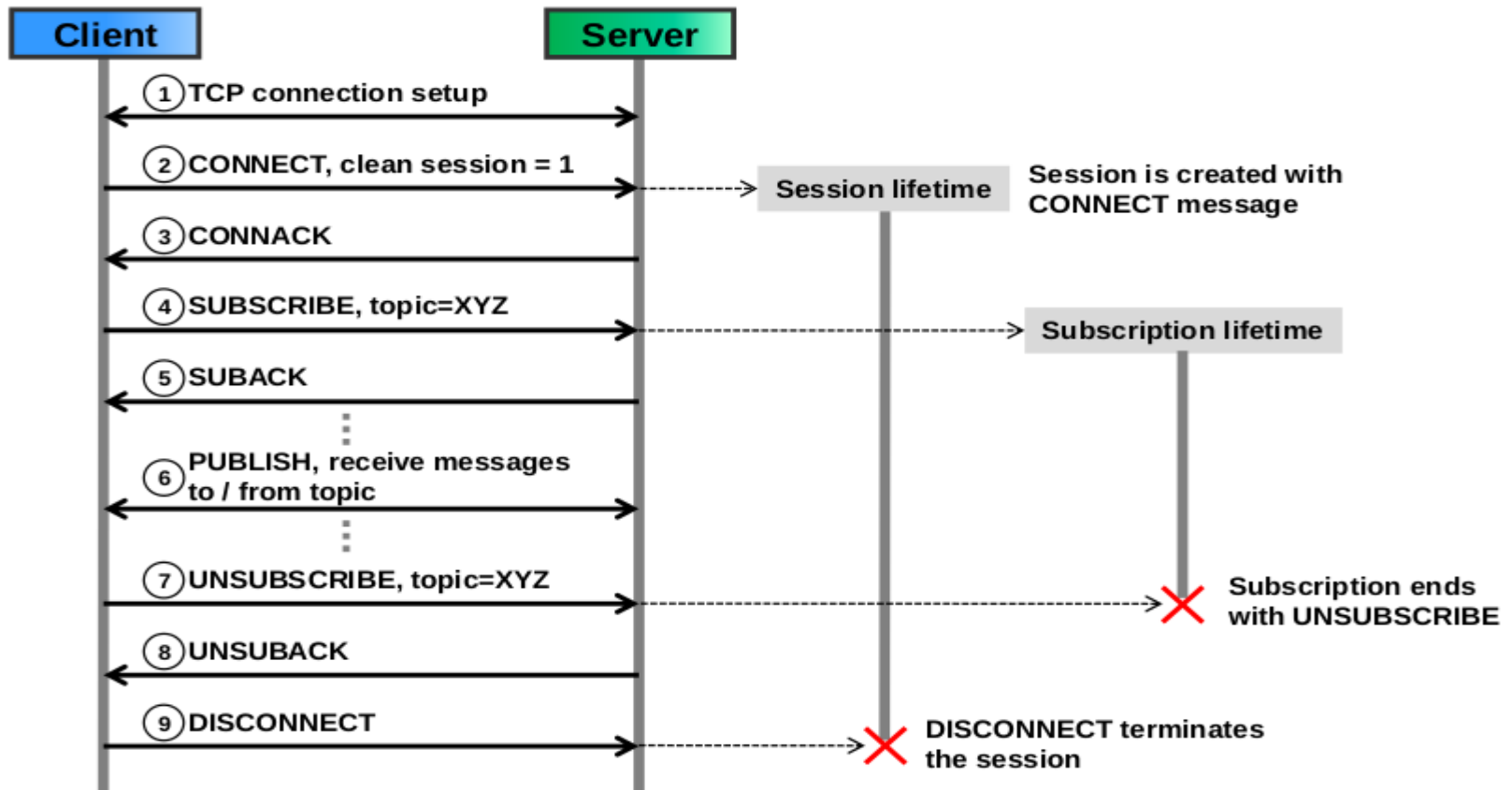
- QoS level 2 affords the highest quality delivery semantics **exactly-once**, but comes with the cost of **additional control messages**.



Example: CONNECT & SUBSCRIBE msg flow

Case 1:

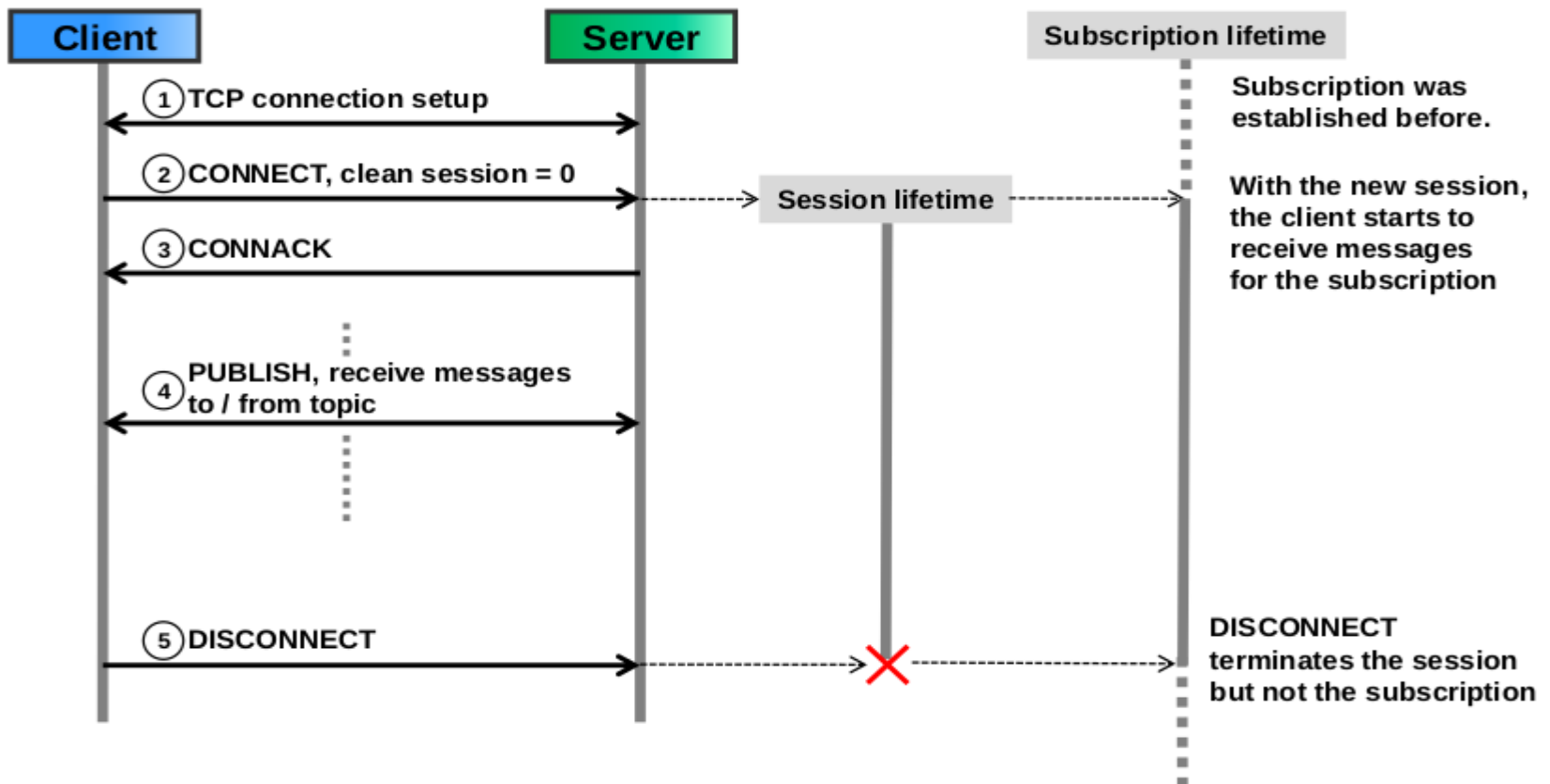
- Session and subscription setup with clean session flag = 1 (transient subscription)



Cont...

Case 2:

- Session and subscription setup with clean session flag = 0 (durable subscription)



Thanks!

